

Azure ML Environments

Method 2: Dockerfile (System + Pip Packages)

When your machine learning job requires OS-level system packages (like `libgl1` for OpenCV, or `build-essential` for compiling C-extensions) in addition to Python packages, using a custom Dockerfile provides the ultimate control over your runtime environment.

Step 1: Directory Structure & Files

Create a dedicated folder for your environment setup containing both your Dockerfile and your Python requirements.

```
env_setup/  
├── requirements.txt  
└── Dockerfile
```

Dockerfile Contents:

```
# Start from a Microsoft base image  
FROM mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04:latest  
  
# Install OS-level system dependencies  
RUN apt-get update && apt-get install -y \  
    libgl1-mesa-glx \  
    build-essential \  
    && rm -rf /var/lib/apt/lists/*  
  
# Copy requirements and install via pip  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt
```

Step 2: End-to-End Submission Example

This script demonstrates how to point the Azure ML SDK to your folder so it can build the Docker image in the cloud, and then submit a job using that exact environment.

```

from azure.identity import DefaultAzureCredential
from azure.ai.ml import MLClient, command
from azure.ai.ml.entities import Environment, BuildContext

# 1. Connect to the Azure ML Workspace
ml_client = MLClient.from_config(credential=DefaultAzureCredential())

# 2. Define Environment via Docker Build Context
docker_env = Environment(
    name="my-custom-docker-env",
    description="Environment built directly from a custom Dockerfile",
    build=BuildContext(
        path="./env_setup" # Path to local folder containing Dockerfile & requirements
    )
)

# Register the environment (Azure ML will queue an ACR image build on first use)
registered_env = ml_client.environments.create_or_update(docker_env)
print(f"Registered Environment: {registered_env.name} v{registered_env.version}")

# 3. Define the Command Job
my_job = command(
    code="./src", # Local folder with your training code
    command="python train.py", # The entry script
    compute="my-cluster-name", # Replace with your actual compute cluster

    # Reference the Docker-based environment
    environment=f"{registered_env.name}:{registered_env.version}",

    display_name="docker-env-training-job"
)

# 4. Submit the Job
submitted_job = ml_client.jobs.create_or_update(my_job)
print(f"Job submitted! View it here: {submitted_job.studio_url}")

```

Note: The very first time you submit a job using a new Dockerfile, Azure ML will take a few minutes in the "Preparing" stage to build the container image. Subsequent jobs will use the cached image and start immediately.